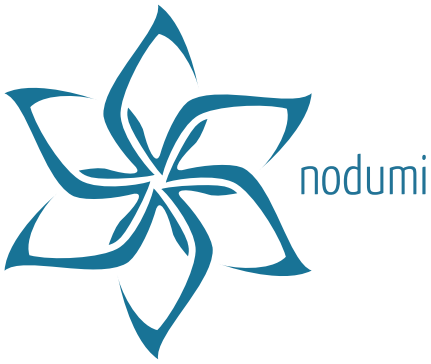
User Manual

`nodumi` – Interactive MIDI Visualizer

26 October 2026



Contents

1 Introduction **3**

2 System Requirements **3**

3 Obtaining Binaries **4**
- 3.1 Prebuilt Binaries . . . 4
- 3.2 Compilation from Source . . . 4

4 Usage **4**
- 4.1 Command Line Interface . . . 4
- 4.2 Graphical Interface . . . 4
- 4.3 Warnings and Errors . . . 4

5 Keyboard Shortcuts **5**
- 5.1 Standard . . . 5
- 5.2 vim-like . . . 6

6 Features **7**
- 6.1 Display Modes . . . 7
  - 6.1.1 Bar . . . 8
  - 6.1.2 Line . . . 8
  - 6.1.3 Pulse . . . 9
  - 6.1.4 Ball . . . 9
  - 6.1.5 FFT . . . 10
  - 6.1.6 Voronoi . . . 11
  - 6.1.7 Loop . . . 11
- 6.2 Sheet Music Generation . . . 12
- 6.3 Song Time . . . 12
  - 6.3.1 Relative . . . 12
  - 6.3.2 Absolute . . . 12
- 6.4 Key Signature Detection . . . 13
- 6.5 Tempo Detection . . . 13

6.6 *Now*-Line . . . 13
6.7 Measure Lines . . . 13
6.8 Measure Numbers . . . 13
6.9 Hide Background . . . 13

7 Real-Time I/O **13**
7.1 Input . . . 13
7.2 Output . . . 14
7.3 Live Play Mode . . . 14
7.4 LSTM-Based Track Separation . . . 14

8 Color Selection **14**
8.1 Color By . . . 14
8.1.1 Part . . . 14
8.1.2 Velocity . . . 14
8.1.3 Tonic . . . 15
8.2 Color Scheme . . . 15
8.2.1 Default . . . 15
8.2.2 From Background . . . 15
8.3 Color Swapping . . . 15
8.4 Color Cycling . . . 15
8.5 Color Inversion . . . 15

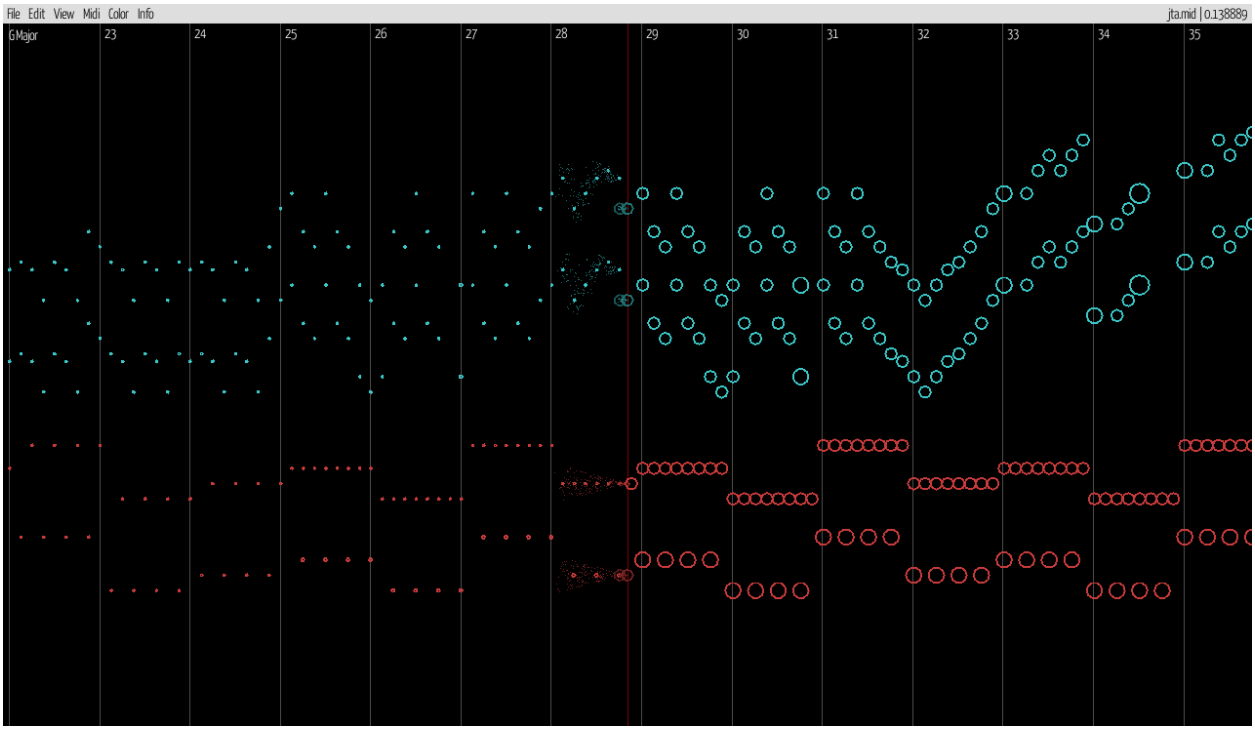
9 The MKI File Format **16**
9.1 Specification . . . 16

10 User Preferences **18**
10.1 Particle Emission . . . 18
10.2 Dynamic Label Coloring . . . 18
10.3 Adaptive Track Division . . . 18
10.4 Hand Range Selection . . . 18
10.5 LSTM Classifier . . . 18
10.6 Quantization Threshold . . . 18
10.7 Image Adjustment . . . 18
10.8 Color Distance Calculation . . . 19
10.9 Display FPS . . . 19
10.10 Limit FPS . . . 19
10.11 Frame-by-Frame Data . . . 19
10.12 Velocity Scaling . . . 19
10.13 Overlay Blending . . . 19
10.14 Shadow Effects . . . 19

11 Frequently Asked Questions **19**

12 Remarks **20**

1 Introduction



`nodumi` is an interactive MIDI visualizer. Both live input (such as from a MIDI-enabled instrument) and prerecorded MIDI files are supported. Moreover, this program defines, implements, and supports a custom file format (MKI) for parametrizing visualization schemes. `nodumi` is inspired by Stephen Malinowski's *Music Animation Machine* (MAM2006), and aims to function as a modernized version of the public-facing MAM2006 program. Many features from MAM2006 are present, and additional features have been added as well. Notably, support for both Windows and many Linux systems is provided, in contrast to the Windows-exclusive nature of MAM2006. Currently, macOS is not supported due to the specific resources required to create builds for the Mac ecosystem. This software is licensed under GPLv3 – therefore the source code is made freely available: (iikare/nodumi).

This application was created by iika-re.

2 System Requirements

For Linux systems, you will need to install `raylib`, either through your distribution's package manager or by directly building and installing the required shared libraries. You will also need the ALSA and JACK libraries (`-lasound -ljack`), and the libraries output by the command `pkg-config --libs gtk+-3.0`. As of 26 October 2026, these are (identified via corresponding linker flag):

```
-lgtk-3 -lgdk-3 -lz -lpangocairo-1.0 -lpango-1.0 -lharfbuzz -latk-1.0 -lcairo-gobject
            -lcairo -lgdk_pixbuf-2.0 -lgio-2.0 -lgobject-2.0 -lglib-2.0
```

For Windows systems, all dependencies are bundled into the executable. A reasonably modern OS is required (testing has only been done on Windows 10, but theoretically earlier Windows versions should be compatible).

A GPU supporting OpenGL 3.3 or higher is required. A relatively modern CPU is desirable, but not necessary. The performance demands of this application are low unless you wish to visualize very large MIDI files. The Voronoi rendering mode is the most demanding, but even integrated GPUs keep a high framerate in it.

3 Obtaining Binaries

This section entails how to obtain `nodumi` – either downloading prebuilt binaries for supported operating systems, or by building directly from source.

3.1 Prebuilt Binaries

At the moment, there is no set release schedule, and prebuilt binaries are no longer kept in the repository. When available, they are provided on the project page. Use `nodumi` for Linux-based systems, and `nodumi.exe` for Windows-based systems. Note that there is *no* installer for either operating system. As there are no persistent configuration files, the application is portable by default. Preferences can be kept by saving them into an MKI file.

3.2 Compilation from Source

NOTE – building from source requires a Linux development environment, as the compilation process depends on tools that are not Windows-compatible. This means that both Linux and Windows executables must be built from Linux.

First, obtain the source code by cloning (iikare/nodumi). Then, run `make rel=y` for a Linux release build, or `make arch=win rel=y` for a Windows release build. Adding `torch=y` to a Linux build enables the LSTM track classifier (see Section 7.4), and `make f` builds both release executables at once. Compile-time dependencies are listed as submodules in the directory, so make sure to obtain them as well. The build also expects `clang`/`clang++` (C++23), the `mold` linker, `zsh`, `git`, `xxd`, `pkg-config`, and the GTK 3, ALSA and JACK development files. Cross-compilation from Linux to Windows requires `x86_64-w64-mingw32-g++`, `x86_64-w64-mingw32-gcc`, and `x86_64-w64-mingw32-ld`. The final executable can be then found in the `bin` directory. This manual is built with `make doc`, which needs `xelatex`, `minted` (with `pygments`) and `inkscape`.

4 Usage

A very brief usage overview is provided below. Further usage information is given in a later section.

4.1 Command Line Interface

While there is no true text-based interface, both a MIDI/MKI file and an image can be loaded automatically through the proper command line arguments. Any order of arguments is accepted, with the first MIDI/MKI file or image file in the list being taken. Any MIDI/MKI or image files past the first of each type and any invalid paths are ignored. MIDI files may end in `.mid` or `.midi`, and images in `.png`, `.jpg` or `.jpeg`. An example is given below.

```
./bin/nodumi [path to MIDI/MKI] [path to image]
```

There are no other command line parameters. After executing the above command, the graphical interface will start automatically.

4.2 Graphical Interface

If not starting the application from the command line, the graphical interface will open automatically without any files loaded. Files can be opened through the `File` menu, or dropped onto the window (up to one MIDI/MKI file and one image at a time).

4.3 Warnings and Errors

Almost all warnings and errors are reported at the command line. Therefore, you will need to view the terminal output to diagnose any potential problems. Log files are not automatically generated on each program run. If you want a method of generating this log file, pipe `stdout` to a temporary log file. This will require you to launch `nodumi` from the command line. An example is shown below.

```
./bin/nodumi [args] > "nodumi_`date +%Y%m%d%H%M%S`.log"
```

5 Keyboard Shortcuts

For ease of use, keyboard shortcuts have been implemented for common actions. They are able to be used both through conventional means as well as through a set of `vim`-like bindings. There is currently no option to rebind shortcuts (of either type).

5.1 Standard

A set of shortcuts (mostly) conforming to conventional norms are defined via the table below.

Key	**Action**
`Ctrl-O`	Open MIDI/MKI File
`Ctrl-Shift-O`	Open Image File
`Ctrl-W`	Close MIDI/MKI File
`Ctrl-Shift-W`	Close Image File
`Ctrl-S`	Save (to MKI)
`Ctrl-Shift-S`	Save As (to MKI)
`Ctrl-R`	Reload MIDI/MKI File
`Ctrl-,`	Open Preferences
`Ctrl-F`	Open File Info
`Ctrl-I`	Open Program Info
`F7`	Exit
`Ctrl-Space`	Toggle Live Play Mode
`Space`	Toggle Playback
`Tab`	Open Preferences
`Ctrl-`$n, n \in [1, 7]$	Enable n^{th} Display Mode
`Home`	Jump to Start
`End`	Jump to End
`Scr.Wheel`	Zoom
`Alt-Scr.Wheel`	Zoom (precise)
$\uparrow \cdot \downarrow$	Zoom
`Alt-`$\uparrow \cdot \downarrow$	Zoom (precise)
$\leftarrow \cdot \rightarrow$	Move Back/Forward
`Ctrl-`$\leftarrow \cdot \rightarrow$	Move Back/Forward (faster)
`Shift-`$\leftarrow \cdot \rightarrow$	Move Back/Forward (by measure)
`Ctrl-Scr.Wheel`	Image Zoom
`Shift-Scr.Wheel`	Image Zoom (precise)

Image zoom only applies while a background image is loaded and shown. The display modes are numbered in the order of the `View` → `Display Mode:` menu (Bar, Line, Pulse, Ball, FFT, Voronoi, Loop).

5.2 vim-like

Similar to the list in Section 5.1, these shortcuts are bound to common actions. These use the command buffer, which is visible at the bottom right corner of the window. Pressing `esc` or enter will clear the buffer. Once a valid command is input to the buffer, it will either be (a) executed immediately if not beginning with `:` or #, or (b) executed after `<Enter>` is pressed by the user. The list of these bindings is presented below.

Binding	Action
`:o`	Open MIDI/MKI File
`:oi`	Open Image File
`:q`	Close MIDI/MKI File
`:qi`	Close Image File
`:r`	Reload MIDI/MKI File
`:qa`	Exit
`:p`	Open Preferences
`:f`	Open File Info
`:n`	Open Program Info
`:l`	Toggle Live Play Mode
`:s`	Toggle Sheet Music Display
`:d`$n, n \in [1, 9]$	Enable n^{th} Display Mode
`:r[w/h]`n	Set Width/Height to n
`:if[w/h]`	Fit Image Width/Height
`gg` or `0`	Jump to Start
`G` or $	Jump to End
n`G`	Jump to n^{th} Measure
`b`	Move Back One Measure
n`b`	Move Back n Measures
`w`	Move Forward One Measure
n`w`	Move Forward n Measures
#`XXXXXX`	Change Selected Color

Note that the color change command expects a 6-digit hex color string as input, and applies to the color selector that is open. For `:d`, values past the last display mode select the last one (Loop). Window sizes set with `:rw` and `:rh` cannot go below the default window size.

6 Features

An explanation of the major features of this program follows.

6.1 Display Modes

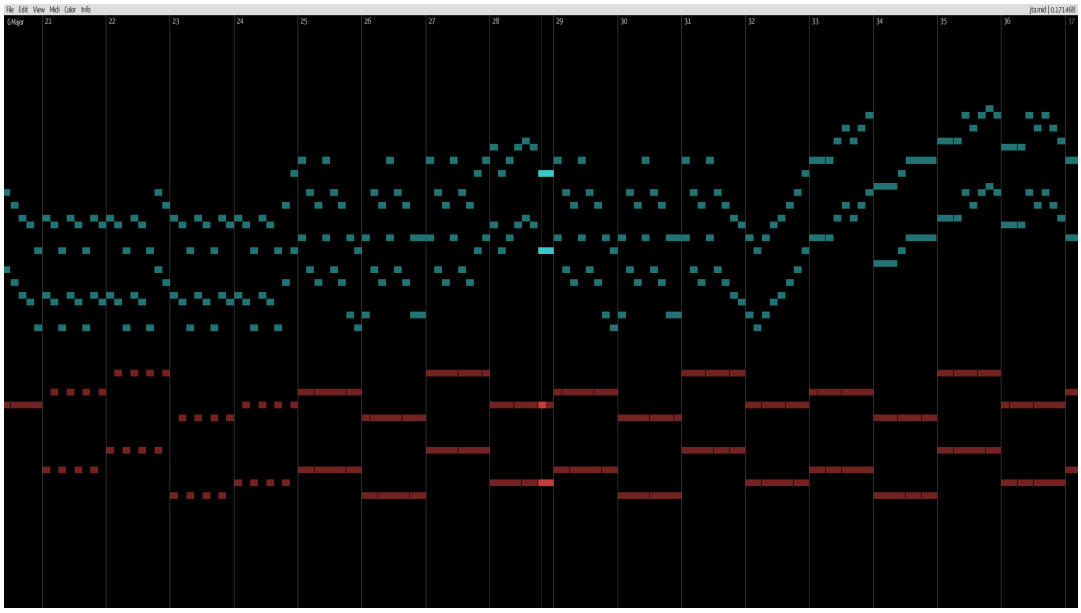
Currently, there are seven fully implemented display modes, all of which are usable for both live input and while using prerecorded files. For all display modes, the x direction represents time, and the y direction represents the 88 keys playable on a standard piano (i.e. pitch). While the MIDI specification provides for note input in the key range 0-127, only values 21-108 are visualized by this application. There are three classes of display modes:

Bar-based	Ignores temporal context of notes.
Line-based	Accounts for temporal context of notes.
Other	Not encapsulated by the above types.

Each mode is marked with its corresponding type. Line-based modes link temporally adjacent notes on the same track.

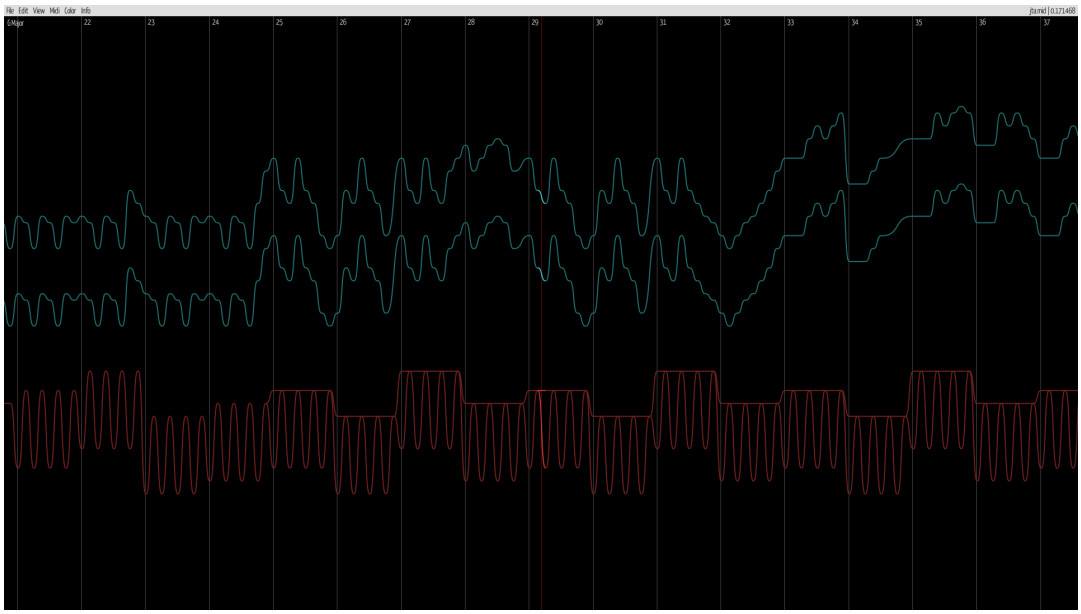
All display modes are accessible via `View` → `Display Mode:` → `[mode]`, and are described below in that menu's order.

6.1.1 Bar



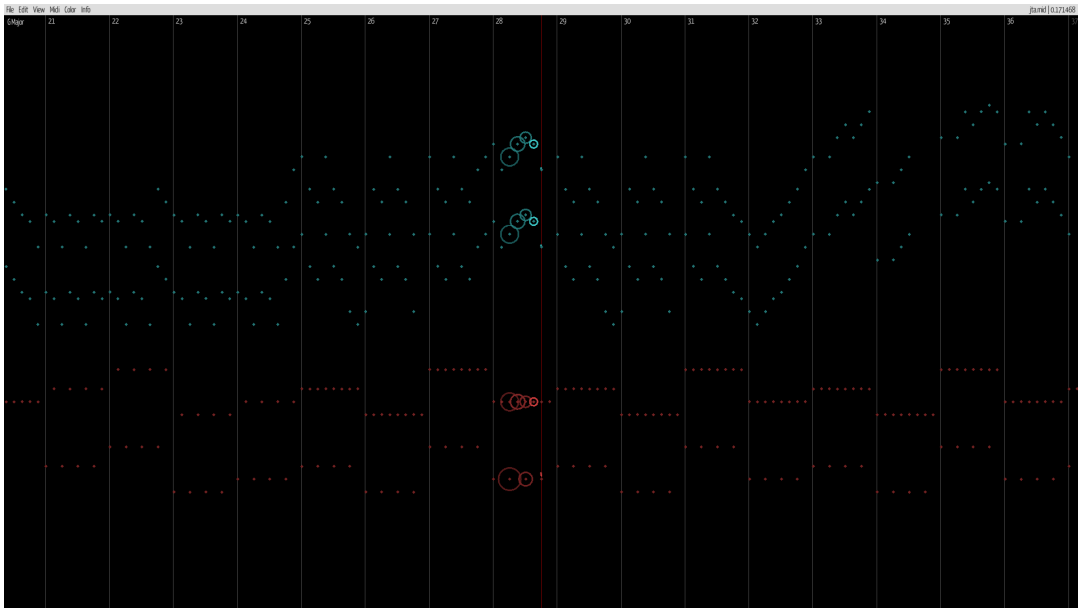
The simplest display mode – this simply draws a bar at the x position of the note with length determined by its duration. This is a BAR-BASED display mode.

6.1.2 Line



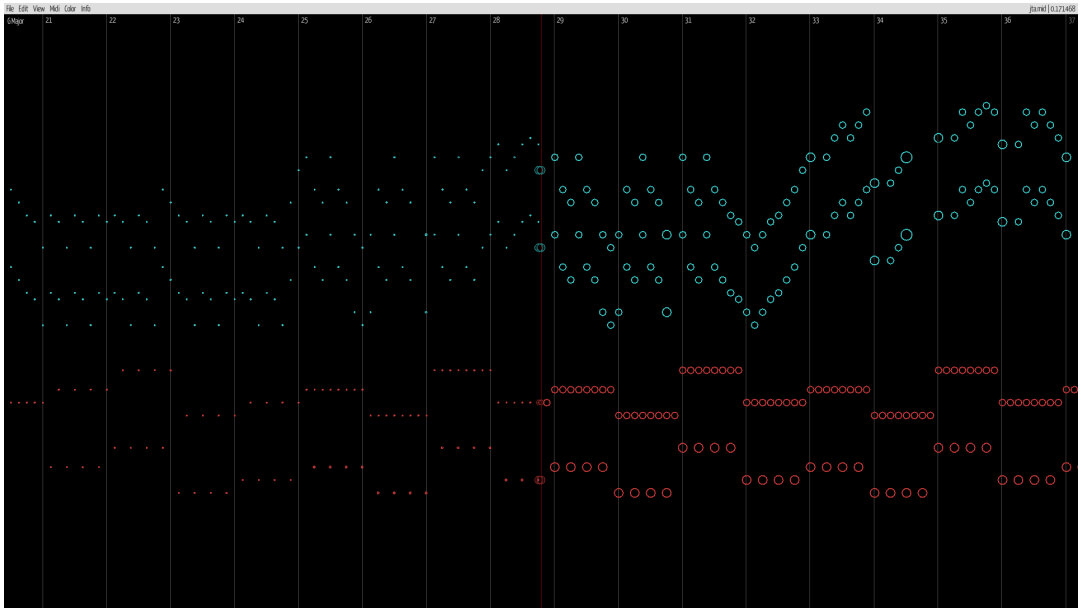
This mode draws a continuous bezier curve between notes. For line segments shorter than a given threshold, a single straight line segment is drawn to optimize performance. This is a LINE-BASED display mode.

6.1.3 Pulse



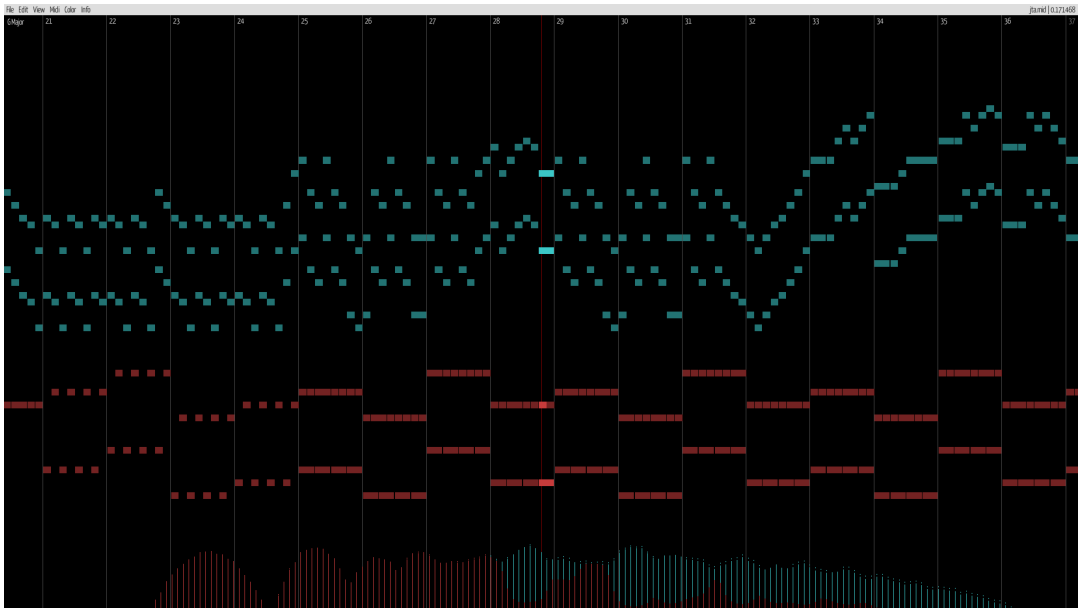
A portion of the straight line between points is drawn instead. The length of this line segment is dependent on the relative position and duration of the note in respect to the *now*-line. Additionally, a pulse appears at a note-on event and fades out over time. This is a LINE-BASED display mode.

6.1.4 Ball



A large circle slowly shrinks as a note is played, and follows the *now*-line. Along with it, a circle is also drawn at the note's original start position, and a line connects the two circles. Once the duration of the note has passed, only a small circle remains at that note's ending position. This is a BAR-BASED display mode.

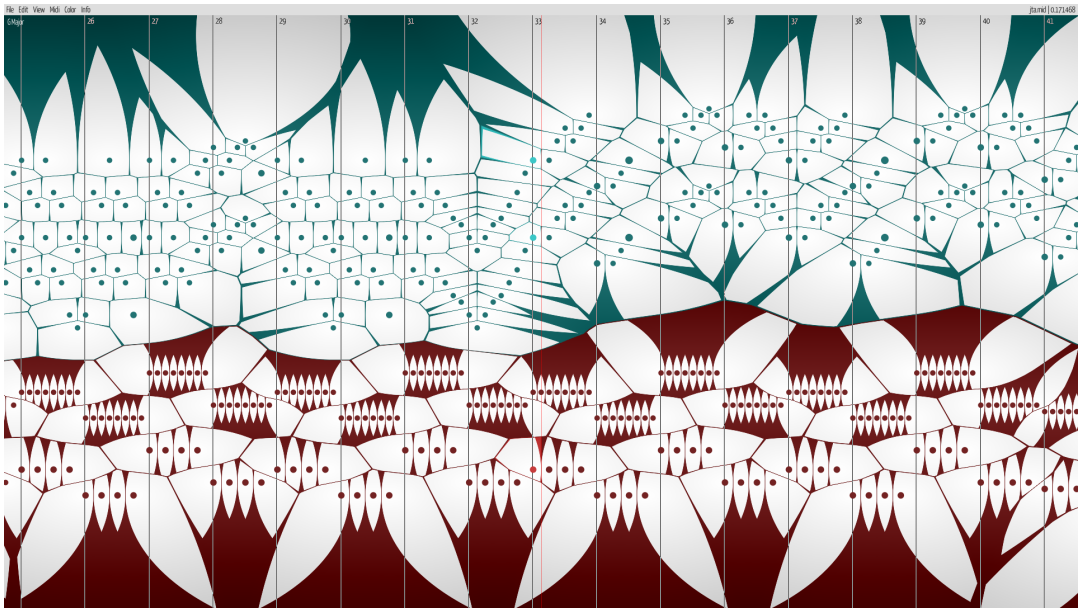
6.1.5 FFT



Identical to the Bar mode, with the addition of a simulated frequency spectrum along the bottom of the screen. Since MIDI data does not contain actual samples, the spectrum is modelled on a piano, even when another MIDI instrument is present on a track. Each sounding note adds its fundamental and overtones (with the slight inharmonicity of real piano strings), which decay faster in the treble than in the bass. A released key is silenced by its damper within about half a second, except for the highest keys (F#6 and up), which have no dampers, and the sustain pedal (CC64) keeps strings ringing as it would on a real instrument.

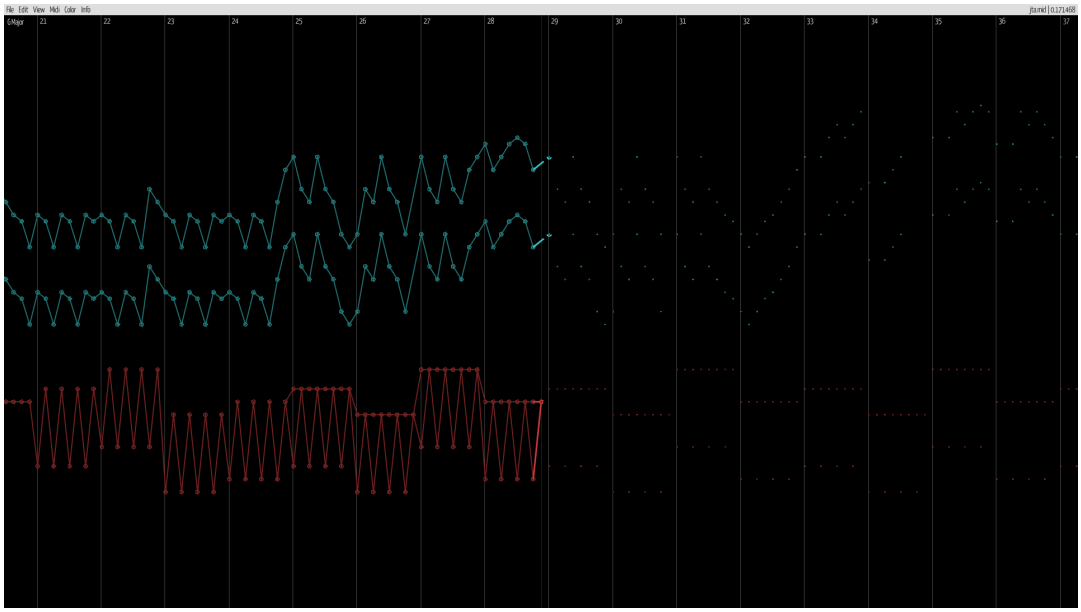
The spectrum is drawn as a bar every 10 pixels, spaced logarithmically in frequency over the audible range of $[20, 20000]$Hz, and showing a range of 66dB. Bars rise immediately and fall gradually, and a small marker briefly holds each bar's recent peak. Every bar is split into the colors of the tracks that make it up, so it is possible to see which part produces each frequency. In practice, not all of the spectrum is used equally (note that C8, the highest key on a piano, has a fundamental of only ~4.2kHz), so most of the upper range is filled by overtones. This is a Bar-based display mode.

6.1.6 Voronoi



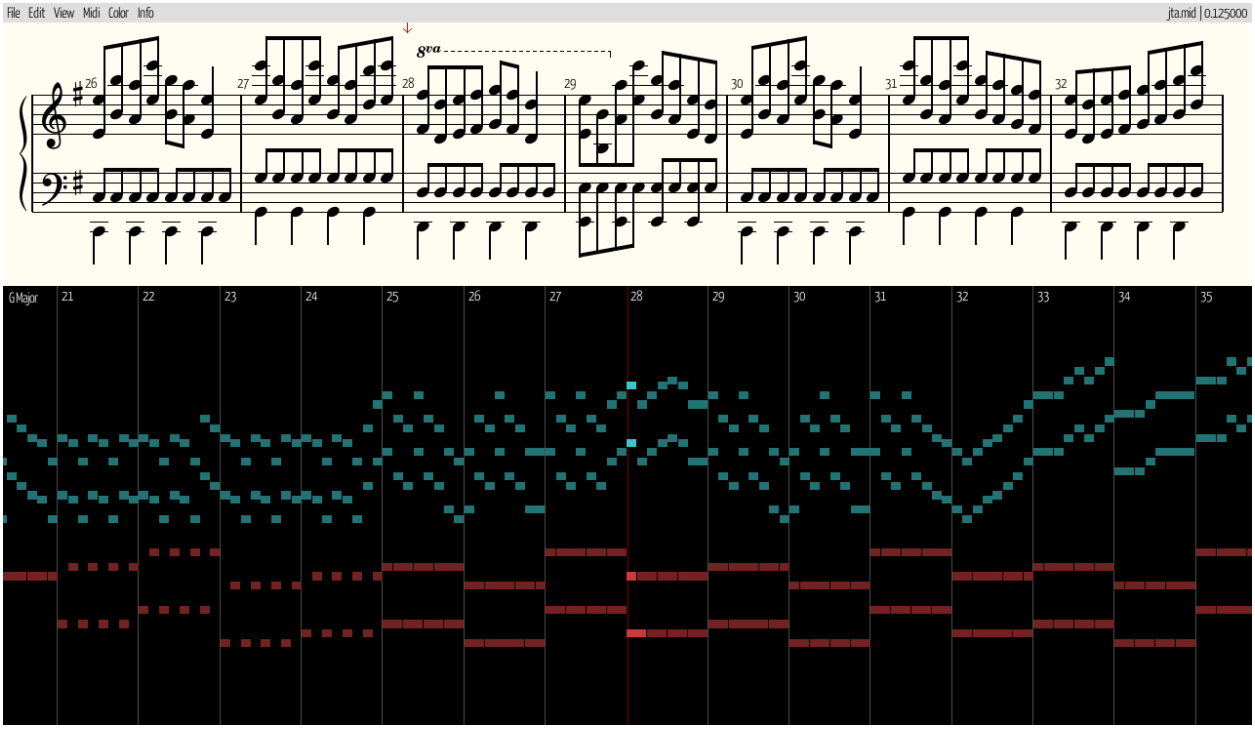
Spatial distance is encoded in a Voronoi-based diagram, calculated in real-time via GPU. Each note's start position is the origin of a Voronoi cell, colored after its note, and marked with a smooth dot sized by the note's duration. Every note within a screen's width of the visible area takes part, so cells at the edges do not change as notes scroll into view. Notes are sorted into a grid before drawing, so the cost of each pixel stays about the same no matter how many notes are shown, and zooming far out remains smooth. Measure lines and numbers are drawn inverted over the cells, so they stay visible on any cell color. Image backgrounds cannot be seen when using this visualization. This is an OTHER display mode.

6.1.7 Loop



Notes on the same track are joined by straight lines, which are drawn in as the music reaches them. Each note is marked by a dot sized by its duration. As the *now*-line moves from one note to the next, the line between them grows and a ring closes around the next note's dot, and the ring stays once that note has been reached. This is a LINE-BASED display mode.

6.2 Sheet Music Generation



The ability to generate sheet music from MIDI/MKI files or from live input is supported. A grand staff is laid out a page at a time above the visualization, and the measure under the *now*-line is marked with an arrow. Notes are split into notatable values per measure, tied across beats and barlines, and given flags or beams grouped by the time signature. Rests fill the gaps, and each stave holds up to two voices with their own stem directions. Dotted values, accidentals, ledger lines, *8va*/*8vb* lines for passages far from the stave, rolled chords and grace notes are also notated.

Since MIDI files record performances rather than notation, note timings are first snapped to a grid. The grid is set separately for files and for live input by the *Quantization Threshold* preferences (see Section 10.6). Files with a single note track are split into two hands as described under Adaptive Track Division. During live input, pages fill from left to right and turn once full, and measures are laid out in 4/4 at a fixed tempo.

This feature is still in development, so dense or unusual passages may still be notated awkwardly.

This feature is accessible via `Edit` → `Enable Sheet Music`.

6.3 Song Time

This option places a marker of the current piece's position in the upper left corner of the visualization, below the menu bar (or below the sheet music, when it is shown). The key signature and tempo are shown next to it. Two options are provided for the format: *Relative Song Time*, and *Absolute Song Time*. For live input, this value is proportional to the amount of time spent in the Live Play mode.

6.3.1 Relative

The *Relative Song Time* option displays a percentage that corresponds to the percentage of the piece complete relative to the *Now*-Line. For live input, this value is not rendered at all instead.

This feature is accessible via `View` → `Show Song Time:` → `Relative`, and hidden via `View` → `Hide Song Time`.

6.3.2 Absolute

The *Absolute Song Time* option displays two numbers – the first one corresponds to the current amount of time elapsed for the current file, while the second is the total length of the file. For live input, this value consists of only one number – the amount of time since the Live Play mode was initiated.

This feature is accessible via `View` → `Show Song Time:` → `Absolute`, and hidden via `View` → `Hide Song Time`.

6.4 Key Signature Detection

For MIDI/MKI files, an option to extract and render the current key signature from the file is provided. In most cases, key signature data is generated directly from the MIDI/MKI file. If an input MIDI file does not have any key signatures, the key signature with the smallest number of off-key notes is used instead. Currently, this assumes the entire piece uses the same key signature. For live input, the key signature that fits the most notes played so far is shown.

This feature is accessible via `View` → `{Show, Hide} Key Signature`.

6.5 Tempo Detection

For MIDI/MKI files, an option to extract and render the current tempo from the file is provided. For live input, this feature is not available.

This feature is accessible via `View` → `{Show, Hide} Tempo`.

6.6 *Now*-Line

A vertical line is displayed at the x-position of the current time marker. This line is static in relation to the notes. It is possible to reposition the line to any valid x-coordinate. To do this, left-click on the line and drag to the intended position. When this is done, the notes will move along with the line as well. In addition, to reset the position of the line, hold `Shift` while left-clicking.

This feature is accessible via `View` → `{Show, Hide} Now Line`.

6.7 Measure Lines

Like the *Now*-Line, vertical lines are placed at the start of each measure. These lines cannot be moved, as they are fixed to the timestamp at which they appear in the MIDI/MKI file. When the Live Play mode is active, measures are counted in 4/4 at a fixed tempo from the moment it is enabled. When zooming out, measure lines fade as they get closer together.

This feature is accessible via `View` → `{Show, Hide} Measure Line`.

6.8 Measure Numbers

With this option, each measure has a corresponding measure number displayed at the top of the screen. The number is rendered slightly to the right of the measure line to prevent overlap. When zooming out, only a subset of measure numbers are shown in order to preserve readability.

This feature is accessible via `View` → `{Show, Hide} Measure Number`.

6.9 Hide Background

If an image is loaded, this option prevents it from being rendered. Otherwise, this option has no effect. A loaded image can be moved by dragging it with the left mouse button, and zoomed with `Ctrl-Scr.Wheel`. Right-clicking the image offers to remove it, or to fit it to the width or height of the window.

This feature is accessible via `View` → `{Show, Hide} Background`.

7 Real-Time I/O

Communication to external MIDI devices is possible. This includes sending MIDI events to a synthesizer, and processing incoming MIDI data from a device. This option also allows the user to play a MIDI instrument and view a live visualization at the same time. When MIDI ports are displayed, a short version of the device-provided port name is provided, and each input or output port is uniquely numbered. The program's own ports are named `nodumi input` and `nodumi output`.

7.1 Input

If an input MIDI port is selected, the program will listen on that port for incoming MIDI messages. Valid messages will be displayed on screen as if they came from a prerecorded file. Note that an input port can be opened even when the Live Play mode is not activated. The program's own output port is left out of the input list, so that it cannot feed its own output back into itself.

This feature is accessible via `Midi` → `Input` → `[port]`.

7.2 Output

If an output MIDI port is selected, the program will output MIDI messages to that port. If a file is loaded, sent messages correspond with the currently playing notes. Pausing or jumping to another position silences any notes that are still sounding. If the Live Input mode is selected (and a valid input port is open), the program will instead act in a MIDI Through regime. Therefore, input notes will still be rendered while allowing for audio to be provided by an external synthesizer.

NOTE: Using virtual MIDI ports on Windows is not supported, and will likely crash the application. This has been reproduced by creating virtual ports with `loopMIDI` and trying to send MIDI messages via those ports.

This feature is accessible via `Midi` → `Output` → `[port]`.

7.3 Live Play Mode

This mode allows live MIDI data to be displayed on screen. To do this, connect a MIDI device to the computer, enable this mode, and then select the corresponding MIDI input port via `Midi` → `Input` → `[port]`.

This feature is accessible via `Midi` → `{Enable, Disable} Live Play`.

7.4 LSTM-Based Track Separation

Using a small LSTM (dependent on `libtorch`, available only on Linux, and requiring compile-time support), left/right hand detection from a single live MIDI input can be done. It is enabled by the *Use LSTM Classifier* preference. This is primarily intended for use with instruments such as piano, organ, etc. If the program is compiled without `libtorch` support, this feature will fall back to a heuristic-based algorithm for track separation, if enabled via the corresponding option.

8 Color Selection

Almost every UI element has a user-controllable color parameter. This section details the ways in which these parameters can be manipulated by the user.

8.1 Color By

This function allows for the coloring of notes via different methods. Each note has two associated colors at all times – one for when it currently playing or selected by the user, and another for all other cases. These options allow setting both of these colors for all notes at the same time using a predefined algorithm. Even when using these options, the user can still manually override any algorithmically generated color.

Colors are changed by right-clicking an object and choosing `Change Color` (or `Change Part Color` for a note, `Change Line Color` for the *now*-line and measure lines), which opens a color selector. While it is open, a hex color can also be entered with the `#XXXXXX` command. The right-click menu of a note also shows its pitch and number at the top, and offers `Set Tonic`.

Object	Controllable Values
Note	{On, Off} Color (for each {Track, Tonic Offset, Velocity Value})
Now-Line	Line Color
Measure Lines	Line Color
Sheet Music	Background Color
Background	Background Color

8.1.1 Part

In this coloring mode, each track has a unique on/off color. In addition, for MIDI/MKI files, the hue of the generated color is determined by ordering each track by its average y value (i.e. the MIDI note #), and assigning a color based on the track's position in this ordering.

This feature is accessible via `Color` → `Color By:` → `Part`.

8.1.2 Velocity

MIDI events provide for each note a velocity value $V \in [0, 127]$. Then, the color is determined by simply mapping the note's velocity to a set of 128 possible colors. This set of colors is evenly-distributed around

the HSV color wheel, and each color only differs in hue.

This feature is accessible via `Color` → `Color By:` → `Velocity`.

8.1.3 Tonic

This colors notes based on the musical concept of *tonic*. The tonic can be set manually via right clicking on a a note and pressing the `Set Tonic` option. This will set the reference tonic as the selected note. Note that you can set the tonic even while this option is not enabled. In that case, you need to enable this option to see your changes.

This feature is accessible via `Color` → `Color By:` → `Tonic`.

8.2 Color Scheme

A color scheme is able to be derived from the provided background image. By default this is disabled (i.e. this option is preset to the `Default` mode). When an image is loaded, the `From Background` mode is available.

8.2.1 Default

The default color scheme. If an image is loaded, it is not used to calculate any element color values.

This feature is accessible via `Color` → `Color Scheme:` → `Default`.

8.2.2 From Background

The note color scheme is determined using the background image. This uses a randomized *k-means* algorithm to pick the amount of colors. Note that the random nature of this process sometimes leads to unsatisfactory results. In that case, simply rerun the operation until the desired color palette has been generated. This operation will generate values for every *Color By* option simultaneously. Using this operation on an image with less than $\max(\#\text{ Tracks}, 12)$ colors is undefined behavior, and is therefore not supported. In this case, the program will emit a warning.

This feature is accessible via `Color` → `Color Scheme:` → `From Background`.

8.3 Color Swapping

This option swaps the note-on and note-off color sets without changing any other color options.

This feature is accessible via `Color` → `Swap Colors`.

8.4 Color Cycling

This option rotates the note-on and note-off color sets one index without changing any other color options.

This feature is accessible via `Color` → `Cycle Colors`.

8.5 Color Inversion

This option inverts the object colors specified by the table below. For any given RGB input color $C = (C_r, C_g, C_b)$ such that $C_r, C_g, C_b \in [0, 255]$, the inverted color C' is defined as $C' = (255 - C_r, 255 - C_g, 255 - C_b)$.

This feature is accessible via `Color` → `Invert Color Scheme`.

Object	**Invertible Values**
Note	{On, Off} Color (for each {Track, Tonic Offset, Velocity Value})
Now-Line	Line Color
Measure Lines	Line Color
Sheet Music	Background Color
Background	Background Color

9 The MKI File Format

Normal MIDI files do not provide the ability to store visualization-related metadata such as color scheme, rendering options, etc. Therefore, the MKI file format has been created to alleviate this problem. This program defines, implements, and provides support for the MKI file format. Notably, every visualization-related parameter accepted by this program is able to be stored in the MKI format. A MIDI bitstream is embedded into the format, and user-provided images are able to be stored as well.

To save a file in MKI format, use the `Ctrl-S`/`Ctrl-Shift-S` keyboard shortcuts, which correspond to the `File` → `Save`/`File` → `Save As` options.

9.1 Specification

The MKI file format is open and non-proprietary – anyone is free to use the specification described below. However, due to the frequently changing nature of this program, this specification is subject to change at any time. This software provides no guarantees that a MKI file created today will be compatible with future versions of the program. Moreover, many of the options are very specific to this program.

The MKI format is defined as follows:

Offset[bits]	**Value**
`0x00[7:4]`	MKI Version (Major)
`0x00[3:0]`	MKI Version (Minor, `1` = `a`)
`0x01[7:7]`	*Now*-Line
`0x01[6:6]`	Display FPS
`0x01[5:5]`	Display Background
`0x01[4:4]`	Display Sheet Music
`0x01[3:3]`	Display Measure Line
`0x01[2:2]`	Display Measure Number
`0x01[1:1]`	Image Exists?
`0x01[0:0]`	Display Key Signature
`0x02[7:7]`	Display Tempo
`0x02[6:0]-0x09`	`[reserved]`
`0x0A[7:4]`	Color Scheme
`0x0A[3:0]`	Display Mode
`0x0B[7:4]`	Song Time Mode
`0x0B[3:0]`	Tonic Offset
`0x0C-0x0F`	`[reserved]`
`0x10-0x2F`	User Preferences (see below)
`0x30-0x3F`	`[reserved]`
`0x40-0x43`	Zoom (float)
`0x44-0x57`	Image Metadata (zero if no image)
`0x58-0x9F`	Tonic Color Data
`0xA0-0x39F`	Velocity Color Data
`0x3A0-0x3A2`	Background Color
`0x3A3-0x3A6`	Track Count (T)
`0x3A7-0x3A6+N`	Track Color Data
next 4 bytes	MIDI Size (M)
next M bytes	Raw MIDI Data
next 4 bytes	Image Format (if image exists)
next 4 bytes	Image Size (P) (if image exists)
next P bytes	Raw Image Data (if image exists)

$N = 6T$, as each color is stored as three bytes (red, green, blue). Color data lists every note-on color first, then every note-off color. Multi-byte values are little-endian. The image metadata consists of the image position (`0x44-0x47`, two 16-bit integers), its scale, default scale and mean value (`0x48-0x53`, three floats), and its color count (`0x54-0x57`). Only tracks containing notes are counted, so MKI files saved by older versions of this program from MIDI data with empty tracks may load with shifted track colors.

Offset[bits]	**User Preference**
0x10[7:7]	Adaptive Track Division (live)
0x10[6:6]	Adaptive Track Division (file)
0x10[5:5]	Custom Hand Range
0x10[4:4]	Darken Background Image
0x10[3:3]	Set CIE Distance Function
0x10[2:2]	Dynamic Label Color
0x10[1:1]	Particle Effects
0x10[0:0]	Scaled Velocities
0x11[7:7]	Drop Shadow
0x11[6:6]	Overlay Blend Mode
0x11[5:5]	Level Control
0x11[4:4]	Limit FPS
0x11[3:3]	Quantization (live)
0x11[2:2]	Quantization (file)
0x12[7:2]	Hand Range (semitones)
0x12[1:0]	CIE Distance Function
0x13	Darken Level
0x14-0x16	Level Control (red, green, blue)
0x17-0x18	Shadow Angle
0x19[7:2]	Shadow Distance
0x19[1:0]	Quantization Grid (live, $\log_2$ of 64^{ths})
0x1A[1:0]	Quantization Grid (file, $\log_2$ of 64^{ths})
0x1B-0x2F	[reserved]

10 User Preferences

This section pertains to the file-agnostic preferences a user can set. All of these options are available via the keyboard shortcuts `Ctrl-,` or `Tab`, which correspond to the `Edit` → `Preferences` action. The preferences are split into three pages (*Display*, *Effects* and *Image*), and are saved along with MKI files.

10.1 Particle Emission

This option controls the particle system. Depending on the selected display mode, particles will be emitted from a predefined location on a note-on event. In all cases, the location of the particle emission is related to the location of the displayed note. A pictorial example is provided in the introduction.

10.2 Dynamic Label Coloring

The label color for measure numbers, key signatures, etc. may not be visible on bright backgrounds. Therefore, this option exists to automatically select the grayscale RGB color $C_{\text{RGB}} = (c, c, c), c \in [0, 255]$ given the background color B_{RGB}, that maximizes $\Delta E(C_{\text{LAB}}, B_{\text{LAB}})$. This color will then be set as the default label color.

10.3 Adaptive Track Division

Many MIDI devices such as digital pianos only support single-channel MIDI output. Then, it can become difficult to discern the hand that plays a specific section of a piece. This option enables an algorithm that processes live MIDI input to split single-channel input into two tracks automatically. Each chord is assigned by weighing the position and span of both hands against the player's hand size, the number of fingers needed, and how often the hands would cross. For files, the algorithm also looks a few chords ahead before deciding. Note that the algorithm output is tuned for use with MIDI-enabled pianos, so using other two-handed (or greater) instruments as input may lead to confusing results.

Note that it is possible to automatically divide one-channel tracks as well. An option for this is present in the user preferences screen. There are separate options for track division – one for live input, and one for MIDI/MKI files. If the corresponding option for MIDI/MKI files is enabled after a single-track MIDI/MKI file is loaded, the user must reload that file for this option to take effect. This option has no effect on multi-track MIDI/MKI files. Only tracks that contain notes are counted, so a file with a tempo track and a single note track is treated as single-track. Both options are enabled by default.

10.4 Hand Range Selection

This option is available only when the Adaptive Track Division option is enabled. The user is able to select the maximum hand range (8th, 9th, 10th, or 11th) that suits them best. The track division algorithm will then take the user's choice into effect. The default hand range is a 10th. This option is unavailable while the LSTM classifier is in use.

10.5 LSTM Classifier

When the program is built with `libtorch` support, live track division can use the LSTM model described in Section 7.4 instead of the heuristic algorithm. This option is unavailable otherwise, or when live track division is disabled.

10.6 Quantization Threshold

Sheet music snaps note timings and durations to a grid of 64th, 32nd, 16th or 8th notes, so that small inaccuracies in a performance do not turn into unreadable rhythms. Nothing shorter than the grid is notated. There are separate options for MIDI/MKI files (default: 32nd notes) and for live input (default: 16th notes), and both can be disabled. Changing the option for files lays out the sheet music again.

10.7 Image Adjustment

If the user-supplied image is too bright, a darkening option has been provided. An additional RGB level control setting also exists for slightly more fine-tuned control over the background image. Note that full image manipulation is beyond the scope of this program, and so if the user has a problem that cannot be solved via this option, they should fix their issue by using an external image editor.

10.8 Color Distance Calculation

This program supports ΔE calculation via three different color distance algorithms: CIE76, CIE94 and CIEDE2000. The user is able to choose freely between these algorithms, which will then be used throughout the entire program. The default color distance algorithm is CIEDE2000. A table summarizing the benefits of each follows.

ΔE Formula	Properties
CIE76	Fastest, Least Accurate
CIE94	Moderate Speed & Accuracy
CIEDE2000	Slowest, Most Accurate

10.9 Display FPS

It simply displays the current FPS at the top right corner, next to the file name and zoom level. Unlike the other options in this section, it is not part of the preferences window.

This feature is accessible via `View` → `{Show, Hide} FPS`.

10.10 Limit FPS

By default, the framerate is limited to the refresh rate of the monitor the window is on. Disabling this option removes the limit, which is mainly useful for measuring performance.

10.11 Frame-by-Frame Data

When this option is enabled, every frame drawn while a file is playing is saved as an image to a `frame` directory in the working directory, named after the file and numbered in order (`frame/[name]_[n].png`). In this mode, playback advances by exactly 1/60 of a second per frame regardless of how long each frame takes to draw, so the images can be joined into a smooth 60 FPS video, for example with:

```
ffmpeg -framerate 60 -i "frame/[name]_%d.png" -pix_fmt yuv420p video.mp4
```

10.12 Velocity Scaling

The default velocity color mode expects velocities in $[0, 127]$. Therefore, if a piece uses a small subset of this range, the contrast between note colors will be low. This option allows the user to map the full color range provided by this color mode to the notes in this file. Currently, this option only works for prerecorded pieces, not live input. If a piece has a velocity range of zero, then this option does nothing and the default velocity coloring will be shown.

10.13 Overlay Blending

The *now*-line can be rendered using the Overlay blending mode (similar to that found in graphic design applications) instead of the selected color.

10.14 Shadow Effects

A drop shadow effect is available to the user. Both the shadow distance and angle are adjustable by the user via the built-in sliders.

11 Frequently Asked Questions

Q – How can I enable audio output?

A – This program does not itself produce audio. This is by the nature of MIDI messages, which do not encode audio samples, but rather note data & metadata. You can enable the MIDI output feature by following the instructions in the Output section, which will allow you to hear note events with an external synthesizer.

Q – I enabled MIDI Input, but there is no visualization shown. How can I fix this?

A – Assuming your MIDI device is working correctly, the most likely explanation is that you have not activated the Live Play Mode.

Q – I enabled MIDI Output, but my audio quality is subpar. How can I fix this?

A – The quality of software MIDI synthesizers greatly varies. Often, a simple way to make a MIDI synthesizer sound far better is to install a custom soundfont. A good free option is `FluidR3_GM.sf2`, available in many package repositories. For Windows, if you are stuck using the terrible `Microsoft GS Wavetable Synth` as your MIDI output device, finding a better alternative is strongly recommended.

Q – Why do some visualizations look different in the Live Play Mode?

A – These visualization modes expect "fully-formed" notes. Since the duration of notes played via live MIDI input cannot be inferred until the note ends, strange behavior often occurs. Even then, all visualization modes still function – albeit to a lesser standard of quality – during live input.

Q – Are languages other than English supported?

A – While support for changing languages exists, there are currently no translations available. Therefore, English is the only supported language. Community translations are welcome, and translation is possible simply by duplicating `LABEL_LANG_EN` in `src/data_str.h`, replacing `EN` with the respective ISO 639-1 language code, and replacing the *value* of each map entry with the correct translated string. Note that the keys *must* be kept in English – they are internal identifiers and are required for the program to function. However, these internal identifiers will not be shown to the end-user.

Q – How is this software licensed and is it free?

A – This program is provided free-of-charge to everyone. Instructions to download the software are provided in the Obtaining Binaries section. The license chosen is the GNU General Public License v3.0 (GPLv3), and the license terms are provided in the repository. As a summary, the end user is guaranteed the freedom to run, study, share, and modify this program subject to the stipulations in the GPLv3 license.

12 Remarks

This program is open-source (iikare/nodumi), and community contributions are welcome. Additional information can be found on my website.

